

Classification de la qualité du vin

Projet 2 – Machine Learning I – ING2-BDML – EFREI 2026/2027

Nakad BELLON et Thomas BELOT

Octobre 2026

Sommaire

1. Introduction	1
2. Prétraitement des données	2
2.1 Statistiques descriptives et valeurs manquantes	2
2.2 Valeurs aberrantes	2
2.3 Encodage et classification binaire	2
2.4 Distribution des classes	3
2.5 Corrélations	4
2.6 Histogrammes	5
2.7 Séparation et normalisation	5
2.8 Sélection de variables	5
3. Modélisation	6
3.1 Choix des modèles	6
3.2 Effet de la normalisation	6
4. Évaluation des modèles	7
4.1 Comparaison des performances	7
4.2 Courbes ROC et AUC	8
4.3 Compromis biais / variance	8
5. Optimisation des hyperparamètres	9
5.1 Protocole	9
5.2 Résultats en validation croisée	9
5.3 Choix du modèle final	9
6. Conclusion	10
7. Bonus réalisés	10

1. Introduction

Objectif. Prédire si un vin rouge est **bon** à partir de ses caractéristiques physico-chimiques. Le problème est binaire : un vin est « bon » (classe 1) si sa note de qualité est **supérieure ou égale à 7**, sinon il est en classe 0.

Données. Jeu *Red Wine Quality* : **1 599 vins rouges, 11 variables** continues (acidité fixe, acidité volatile, acide citrique, sucre résiduel, chlorures, SO₂ libre, SO₂ total, densité, pH, sulfates, alcool) et une note de qualité de 0 à 10.

Démarche. Prétraitement (section 2), modélisation (section 3), évaluation (section 4), puis optimisation des hyperparamètres (section 5). Le code complet est dans le notebook `projet_qualite_vin.ipynb`.

2. Prétraitement des données

2.1 Statistiques descriptives et valeurs manquantes

Les variables sont d'échelles très différentes (chlorures : 0,087 g/L en moyenne ; SO₂ total : 46,5 mg/L). La qualité varie de 3 à 8 ; la plupart des vins sont notés 5 (681 vins) ou 6 (638 vins). Le jeu ne contient **aucune valeur manquante**.

2.2 Valeurs aberrantes

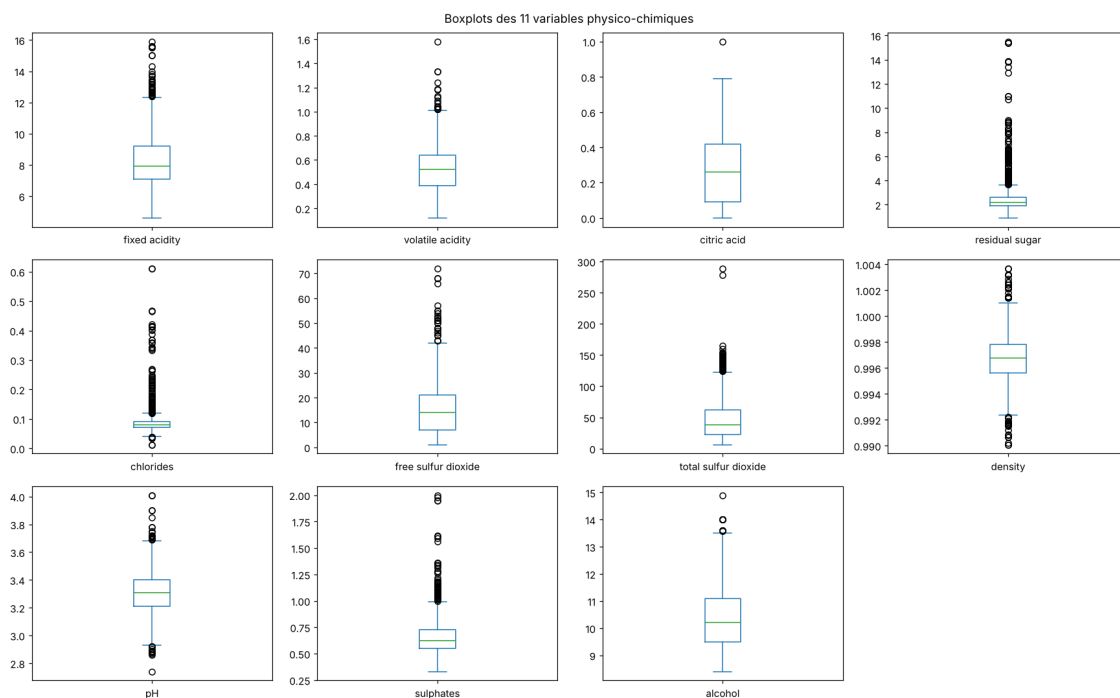


Figure 1: Boxplots des 11 variables

25,3 % des vins ont au moins une valeur au-delà des moustaches, surtout pour le sucre résiduel (155 vins), les chlorures (112), les sulfates (59) et le SO₂ total (55). Ces variables sont asymétriques et les valeurs restent plausibles : **elles sont conservées**.

2.3 Encodage et classification binaire

Toutes les variables sont numériques : **aucun encodage** n'est nécessaire. La cible `bon_vin` vaut 1 si `quality` $\geq$ 7, puis la note est retirée des variables explicatives.

2.4 Distribution des classes

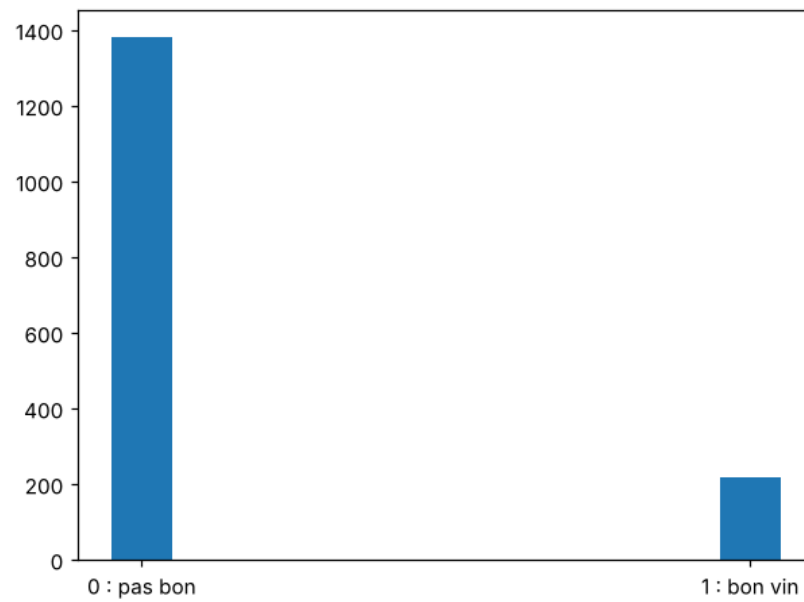


Figure 2: Nombre de vins par classe

Le jeu est **déséquilibré** : **13,6 %** de bons vins (217 sur 1 599). Un modèle qui répond toujours « pas bon » obtient déjà 86 % d'accuracy. L'accuracy seule est donc trompeuse : les modèles sont comparés avec la **moyenne de l'accuracy et du rappel**.

2.5 Corrélations

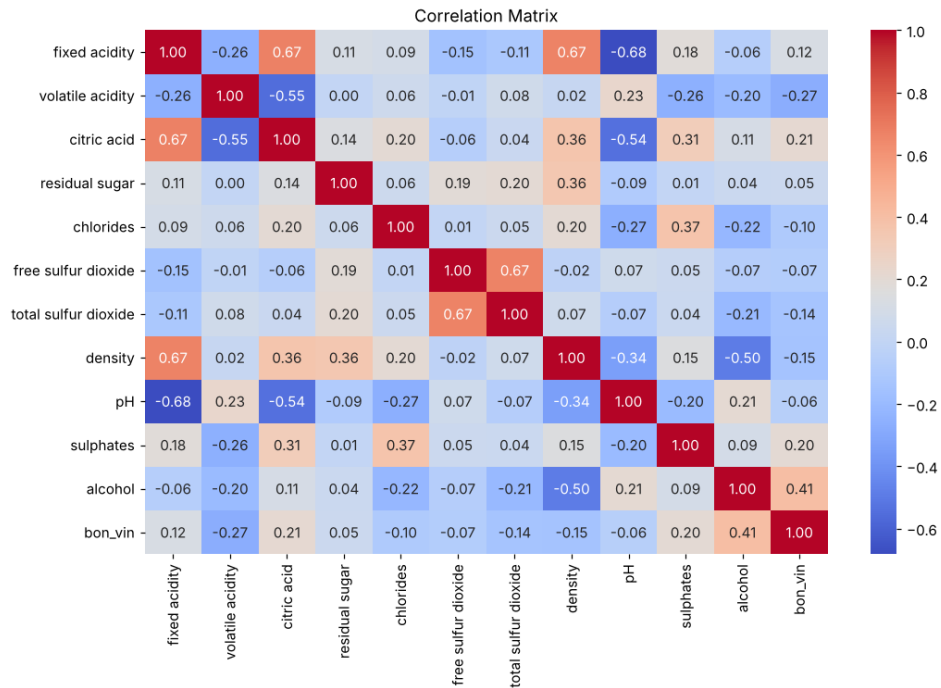


Figure 3: Matrice de corrélation

Tableau 1 - Variables les plus corrélées à la qualité

Variable	Corrélation avec bon_vin
alcohol	+0,41
volatile acidity	-0,27
citric acid	+0,21
sulphates	+0,20
density	-0,15
total sulfur dioxide	-0,14

Le sucre résiduel (+0,05), le pH (-0,06) et le SO₂ libre (-0,07) n'ont presque pas de lien avec la qualité. Plusieurs variables sont liées entre elles : l'acidité fixe avec l'acide citrique (+0,67), la densité (+0,67) et le pH (-0,68), et le SO₂ libre avec le SO₂ total (+0,67). Ces corrélations contredisent l'hypothèse d'indépendance de Naive Bayes.

2.6 Histogrammes

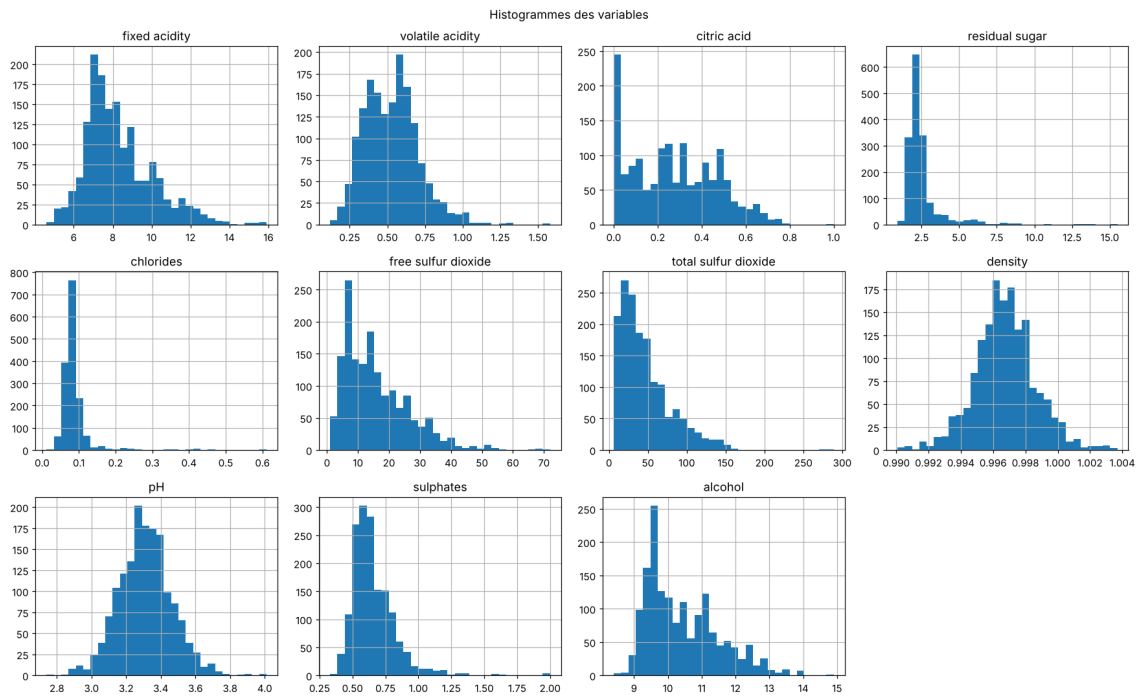


Figure 4: Histogrammes des variables

Plusieurs variables sont asymétriques (sucre résiduel, chlorures, SO₂, sulfates) : l'hypothèse gaussienne de Naive Bayes n'est pas respectée.

2.7 Séparation et normalisation

Les données sont séparées en **1 199 vins** d'apprentissage et **400** de test (25 %, random_state=1). Le test contient 45 bons vins. Les variables sont normalisées avec StandardScaler, ajusté sur l'apprentissage seul.

2.8 Sélection de variables

Trois familles de méthodes sont appliquées à l'apprentissage :

- **Filter** : Pearson, Chi², information mutuelle ;
- **Wrapper** : élimination récursive (RFE) ;
- **Embedded** : Lasso et Ridge.

Chaque méthode retient 6 variables sur 11 (les coefficients non nuls pour le Lasso). On garde les variables retenues par au moins 3 méthodes sur 6.

Tableau 2 - Nombre de méthodes qui retiennent chaque variable (sur 6)

Variable	Votes	Variable	Votes
volatile acidity	6	fixed acidity	3
alcohol	6	chlorides	3
sulphates	6	density	3
total sulfur dioxide	5	pH	1
citric acid	4	free sulfur dioxide	0

Variable	Votes	Variable	Votes
		residual sugar	0

8 variables sont retenues. Le pH, le SO2 libre et le sucre résiduel, peu liés à la qualité, sont écartés. La suite utilise ces 8 variables.

3. Modélisation

3.1 Choix des modèles

Tableau 3 - Modèles testés

Modèle	Linéaire ?	Justification
Arbre de décision (entropy)	Non	Insensible à l'échelle et aux valeurs atypiques ; risque de sur-apprentissage
Naive Bayes	Non	Simple, mais suppose des variables indépendantes et gaussiennes
Régression logistique	Oui	Frontière linéaire, sortie probabiliste
k-NN (k = 1, 3, 5, 7)	Non	Basé sur une distance : normalisation nécessaire, sensible au bruit
SVM linéaire	Oui	Hyperplan de marge maximale
SVM rbf, sigmoid, poly (degré 2)	Non	Frontière non linéaire grâce au noyau

Une fonction entraîne chaque modèle et calcule l'accuracy, le rappel et leur moyenne, ainsi que la précision, le F1 et l'AUC.

3.2 Effet de la normalisation

Tableau 4 - Moyenne accuracy + rappel sur le test

Modèle	Sans normalisation	Avec normalisation
Arbre de décision	0,726	0,728
Naive Bayes	0,774	0,771
Régression logistique	0,603	0,588
KNN (K=1)	0,656	0,735
KNN (K=3)	0,631	0,636
KNN (K=5)	0,639	0,727
KNN (K=7)	0,604	0,677
Linear SVM	0,444	0,444
RBF SVM	0,456	0,608
Sigmoid SVM	0,444	0,571
Polynomial (2) SVM	0,444	0,444

- Le k-NN et les SVM rbf et sigmoid progressent : ils dépendent des distances.

- L'arbre et Naive Bayes ne changent presque pas.
- Le SVM linéaire et le SVM polynomial prédisent « pas bon » pour tous les vins, avec ou sans normalisation.

La suite utilise les données normalisées.

4. Évaluation des modèles

4.1 Comparaison des performances

Tableau 5 - Modèles de base sur le test (données normalisées)

Modèle	Accuracy	Précision	Rappel	F1	AUC	Moy. acc+rappel
Naive Bayes	0,830	0,368	0,711	0,485	0,839	0,771
KNN (K=1)	0,870	0,443	0,600	0,509	0,752	0,735
Arbre de décision	0,878	0,464	0,578	0,515	0,747	0,728
KNN (K=5)	0,898	0,543	0,556	0,549	0,881	0,727
KNN (K=7)	0,888	0,500	0,467	0,483	0,870	0,677
KNN (K=3)	0,872	0,429	0,400	0,414	0,793	0,636
RBF SVM	0,905	0,667	0,311	0,424	0,841	0,608
Régression logistique	0,888	0,500	0,289	0,366	0,864	0,588
Sigmoid SVM	0,830	0,275	0,311	0,292	0,726	0,571
Linear SVM	0,888	0,000	0,000	0,000	0,849	0,444
Polynomial (2) SVM	0,888	0,000	0,000	0,000	0,718	0,444

- **Naive Bayes** est le meilleur (0,771) grâce à son rappel : il détecte 32 bons vins sur 45.
- L'accuracy est trompeuse : le SVM linéaire et le SVM polynomial obtiennent 0,888 sans détecter aucun bon vin, comme un modèle qui répondrait toujours « pas bon » (355 vins sur 400).
- Le SVM rbf et la régression logistique sont précis mais détectent peu de bons vins (rappel $\approx 0,3$).

4.2 Courbes ROC et AUC

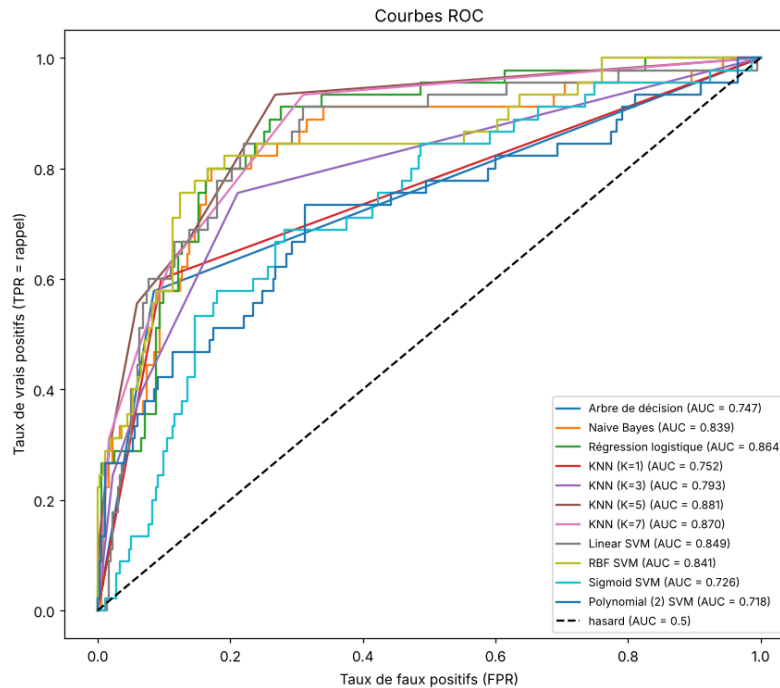


Figure 5: Courbes ROC des 11 modèles

Les meilleures AUC sont celles du k-NN avec $k = 5$ (0,881) et $k = 7$ (0,870), et de la régression logistique (0,864). La régression logistique et le SVM linéaire ont une bonne AUC mais un rappel faible : c'est le seuil de décision qui pénalise les bons vins, pas le score du modèle.

4.3 Compromis biais / variance

Tableau 6 - Moyenne accuracy + rappel sur l'apprentissage et sur le test

Modèle	Train	Test	Écart
Naïve Bayes	0,766	0,771	-0,005
KNN (K=1)	1,000	0,735	0,265
Arbre de décision	1,000	0,728	0,272
KNN (K=5)	0,752	0,727	0,025
KNN (K=7)	0,712	0,677	0,035
KNN (K=3)	0,833	0,636	0,197
RBF SVM	0,621	0,608	0,013
Régression logistique	0,594	0,588	0,006
Sigmoid SVM	0,552	0,571	-0,019
Linear SVM	0,428	0,444	-0,016
Polynomial (2) SVM	0,428	0,444	-0,016

- **Arbre et k-NN (k = 1)** : score parfait sur l'apprentissage et écart d'environ 0,27 → **sur-apprentissage** (variance).
- **k-NN** : l'écart diminue quand k augmente.
- **Régression logistique et SVM** : écart proche de 0 mais scores faibles → **biais**.
- **Naïve Bayes** : écart nul et meilleur score → meilleur compromis parmi les modèles de base.

5. Optimisation des hyperparamètres

5.1 Protocole

- **Grid Search** pour la régression logistique (C, class_weight) et le SVM (noyau, C, class_weight).
- **Random Search** (30 tirages) pour l'arbre (max_depth, min_samples_split, min_samples_leaf, class_weight) et le k-NN (k, distance euclidienne ou de Manhattan, pondération par la distance).
- Validation croisée à **5 plis** sur l'apprentissage ; critère : moyenne de l'accuracy et du rappel.
- class_weight='balanced' donne plus de poids aux bons vins (classe minoritaire).
- Naive Bayes est utilisé sans hyperparamètre.

5.2 Résultats en validation croisée

Tableau 7 - Modèles optimisés (moyenne accuracy + rappel)

Modèle	Meilleurs hyperparamètres	CV	Écart-type	Train	Écart train – CV
Arbre de décision	max_depth = 8, min_samples_split = 15, min_samples_leaf = 1, class_weight = balanced	0,798	0,064	0,934	0,136
Régression logistique	C = 10, class_weight = balanced	0,796	0,032	0,796	0,000
SVM	noyau linéaire, C = 1, class_weight = balanced	0,791	0,045	0,797	0,006
k-NN	k = 12, Manhattan, pondéré par la distance	0,745	0,051	1,000	0,255

Pour le SVM, le meilleur score par noyau est de 0,791 (linéaire), 0,784 (rbf), 0,742 (sigmoid) et 0,690 (poly).

- class_weight='balanced' est retenu par les trois modèles qui l'acceptent.
- Régression logistique et SVM : même score en apprentissage et en validation → pas de variance.
- L'arbre (écart 0,136) et le k-NN (écart 0,255) sur-apprennent encore.
- Les trois meilleurs (0,791 à 0,798) sont équivalents : leurs différences sont plus petites que les écarts-types (0,032 à 0,064).

5.3 Choix du modèle final

Parmi les modèles à moins d'un écart-type du meilleur ($0,798 - 0,064 = 0,734$), on garde celui dont l'écart entre apprentissage et validation est le plus faible (compromis biais / variance). Le modèle retenu est la **régression logistique** (C=10, class_weight='balanced'). Le test n'est utilisé qu'une seule fois, pour ce modèle.

Tableau 8 - Matrice de confusion du modèle final sur le test

	Prédit « pas bon »	Prédit « bon »
Réel « pas bon » (355)	274	81
Réel « bon » (45)	7	38

Tableau 9 – Performances du modèle final

Accuracy	Précision	Rappel	F1	AUC	Moy. acc+rappel test	Moy. acc+rappel train
0,780	0,319	0,844	0,463	0,869	0,812	0,796

- Le modèle détecte **38 bons vins sur 45** (rappel 0,844), au prix de 81 fausses alertes (précision 0,319).
- Le score de test (0,812) est proche de celui de l'apprentissage (0,796) : pas de sur-apprentissage.
- Le test ne contient que 45 bons vins : ces métriques restent incertaines.

6. Conclusion

Étape	Résultat
Prétraitement	Aucune valeur manquante ni variable catégorielle ; valeurs atypiques conservées ; 13,6 % de bons vins
Sélection de variables	8 variables retenues sur 11
Normalisation	Indispensable pour le k-NN et les SVM
Modèles de base	Naive Bayes le meilleur (0,771) ; arbre et k-NN (k = 1) en sur-apprentissage ; modèles linéaires biaisés
Optimisation	class_weight='balanced' corrige le déséquilibre des classes
Modèle final	Régression logistique : 0,812 sur le test, rappel de 0,844

Le déséquilibre des classes est le point central du projet. Avec class_weight='balanced', le rappel de la régression logistique passe de 0,289 à 0,844 : elle devient le modèle le plus stable et détecte la grande majorité des bons vins.

Limites. La qualité est une note de dégustateurs, que les mesures physico-chimiques n'expliquent qu'en partie. Le test ne contient que 45 bons vins. Enfin, le modèle final a une précision faible : environ deux tiers des vins qu'il classe « bons » ne le sont pas.

7. Bonus réalisés

Bonus	Réalisation
Dépôt GitHub	Lien : [à compléter]
Tests unitaires	10 tests pytest : chargement, valeurs manquantes, cible, séparation, sélection, métriques, résultats du modèle final, prédiction
Conteneurisation	Dockerfile : installe les dépendances, entraîne le modèle et démarre l'application
Application web	app.py (Streamlit) : saisie des 8 mesures d'un vin et prédiction « bon vin » ou « pas bon »